

UniApp uni.request 各类参数详解与实战指南

在UniApp跨平台开发中，`uni.request` 作为发起网络请求的核心API，其参数配置直接决定请求的发起方式、数据格式、响应处理逻辑及多端兼容性。这些参数可划分为**基础配置参数**、**请求内容参数**、**响应处理参数**、**安全与超时参数**四大类。本文将逐一拆解每类参数的核心作用、取值范围及实战技巧，帮助开发者精准配置请求参数，避免常见问题。

一、参数分类总览

uni.request的参数以对象形式传入，各类参数的核心定位与关联关系如下：

- 基础配置参数**：定义请求的核心标识（如URL、请求方法），是发起请求的必填项。
- 请求内容参数**：携带请求数据（如查询参数、表单数据），决定传递给服务端的信息。
- 响应处理参数**：控制响应数据的解析方式（如数据类型、响应头），适配不同服务端返回格式。
- 安全与超时参数**：保障请求安全（如请求头令牌）、控制请求生命周期（如超时时间），提升请求稳定性。



参数优先级规则：同一配置项，请求参数中定义的内容会覆盖全局配置（如manifest.json中的网络配置）；必填参数缺失会直接导致请求失败，需重点关注。

二、基础配置参数：请求的“身份标识”

基础配置参数是发起请求的核心，包含URL、请求方法等必填项，直接决定请求的目标地址与交互方式。

1. url：请求目标地址（必填）

定义服务端接口的完整地址或相对路径，是uni.request唯一的必填参数。

核心规则

- 支持绝对路径（如 `https://api.example.com/user/info`）和相对路径（如 `/user/info`），相对路径需配合baseURL全局配置使用。
- H5端需注意跨域问题，开发环境可通过manifest.json的h5.devServer配置代理，生产环境需服务端配置CORS。

- URL中若包含特殊字符（如中文、空格），需用 `encodeURIComponent` 编码，避免请求地址无效。

实战示例

Code block

```
1 // 1. 绝对路径（推荐，兼容性好）
2 uni.request({
3   url: 'https://api.example.com/goods/list'
4 })
5
6 // 2. 相对路径（需在main.js中配置baseUrl）
7 // main.js全局配置
8 uni.$http.setConfig({
9   baseUrl: 'https://api.example.com'
10 })
11 // 页面中使用相对路径
12 uni.request({
13   url: '/goods/list' // 实际请求地址: https://api.example.com/goods/list
14 })
15
16 // 3. 含特殊字符的URL（中文编码）
17 const keyword = '手机'
18 const url = `https://api.example.com/search?
19   keyword=${encodeURIComponent(keyword)}`
20 uni.request({ url })
```

2. method：请求方法（可选，默认GET）

定义HTTP请求方法，决定服务端对请求数据的处理逻辑（如查询、新增、修改）。

核心取值与场景

请求方法	核心作用	数据传递方式	适用场景
GET	查询数据	数据拼接在URL后（query参数）	商品列表查询、详情
POST	提交数据（新增/修改）	数据放在请求体（body）中	用户登录、表单提
PUT	全量更新数据	请求体（body）	修改用户完整信息
DELETE	删除数据	URL参数或请求体	删除商品、取消订

实战注意事项

- GET请求的data参数会自动转化为URL查询参数，无需手动拼接。
- 部分服务端不支持PUT/DELETE方法，可改用POST并在请求头中指定 `X-HTTP-Method-Override`。
- 小程序端对请求方法的大小写不敏感，但推荐使用大写（如GET而非get），保持规范。

3. dataType：响应数据类型（可选，默认json）

指定服务端返回数据的解析方式，决定success回调中res.data的格式。

核心取值

- `json`（默认）：将响应数据解析为JSON对象，若解析失败则返回原始字符串。
- `text`：不解析响应数据，直接返回原始字符串（适用于HTML、XML等非JSON格式）。
- `jsonp`：用于跨域请求，通过JSONP方式获取数据（仅H5端支持）。

实战示例

Code block

```
1  // 1. 解析为JSON（默认，无需配置）
2  uni.request({
3    url: 'https://api.example.com/user/info',
4    success: (res) => {
5      console.log(typeof res.data) // object
6    }
7  })
8
9  // 2. 接收HTML字符串（指定dataType为text）
10 uni.request({
11   url: 'https://api.example.com/page/html',
12   dataType: 'text',
13   success: (res) => {
14     console.log(typeof res.data) // string (HTML内容)
15   }
16 })
17
18 // 3. H5端JSONP跨域请求
19 uni.request({
20   url: 'https://api.example.com/cross-domain',
21   dataType: 'jsonp',
22   jsonp: 'callback', // 自定义回调函数名（默认callback）
23   success: (res) => {
24     console.log(res.data) // JSON对象
```

```
25     }
26   })
```

三、请求内容参数：传递给服务端的“核心信息”

请求内容参数用于携带业务数据，包括data（请求数据）、header（请求头）等，其格式需与服务端接口要求一致。

1. data：请求数据（可选）

携带请求的业务数据，格式需根据请求方法和content-type灵活调整，是前后端数据交互的核心。

数据格式与content-type匹配规则

data的格式由header中的Content-Type 决定，服务端需根据该字段解析数据，常见匹配关系如下：

Content-Type	data格式	适用场景	示例
application/json （默认）	JSON对象	POST/PUT请求，复杂数据（对象、数组）	{ id: 1, name: "张三" }
application/x-www-form-urlencoded	JSON对象（自动转为key=value&key=value）	POST请求，简单表单数据	id=1&name=张三
multipart/form-data	FormData对象	文件上传或混合表单数据	包含文件流和普通数据

实战示例

Code block

```
1  // 1. JSON格式数据 (默认Content-Type: application/json)
2  uni.request({
3    url: 'https://api.example.com/user/save',
4    method: 'POST',
5    data: {
6      id: 1,
7      name: '张三',
8      tags: ['前端', 'UniApp'] // 支持数组类型
9    },
10   success: (res) => {
11     console.log('保存成功', res.data)
12   }
13 })
14
```

```

15 // 2. 表单格式数据 (Content-Type: application/x-www-form-urlencoded)
16 uni.request({
17   url: 'https://api.example.com/user/login',
18   method: 'POST',
19   header: {
20     'Content-Type': 'application/x-www-form-urlencoded'
21   },
22   data: {
23     username: 'admin',
24     password: '123456'
25   },
26   success: (res) => {
27     console.log('登录结果', res.data)
28   }
29 })
30
31 // 3. 混合表单数据 (文件+普通字段, Content-Type: multipart/form-data)
32 const formData = new FormData()
33 formData.append('username', '张三')
34 formData.append('avatar', fileObject) // fileObject为uni.chooseImage获取的文件对象
35
36 uni.request({
37   url: 'https://api.example.com/user/upload',
38   method: 'POST',
39   header: {
40     'Content-Type': 'multipart/form-data'
41   },
42   data: formData,
43   success: (res) => {
44     console.log('上传结果', res.data)
45   }
46 })

```

2. header：请求头（可选）

携带请求的元信息（如身份令牌、数据格式声明），是服务端验证请求合法性、解析数据的重要依据。

常用请求头字段

- `Content-Type`：声明请求数据格式（如application/json），默认值为application/json。
- `Authorization`：携带身份验证信息（如JWT令牌），格式通常为 `Bearer <token>`。
- `User-Agent`：声明客户端信息（如设备型号、应用版本），部分服务端用于统计或适配。

- `Accept`：声明客户端可接收的响应数据格式（如application/json, text/html）。

实战注意事项

- 自定义请求头字段需避免使用下划线（如 `user_id`），部分浏览器或服务端可能无法识别，建议用连字符（`user-id`）。
- 小程序端有默认请求头（如 `Referer`），自定义头不会覆盖默认值，除非明确重写。
- 携带token时，需先从缓存中获取，确保用户已登录：

Code block

```
1  // 携带JWT令牌的请求头配置
2  const token = uni.getStorageSync('token')
3  uni.request({
4    url: 'https://api.example.com/user/info',
5    header: {
6      'Authorization': `Bearer ${token}`, // 身份验证
7      'Content-Type': 'application/json', // 数据格式声明
8      'App-Version': '1.0.0' // 自定义应用版本头
9    },
10   success: (res) => {
11     console.log('用户信息', res.data)
12   }
13 })
```

四、响应处理参数：控制请求结果的“解析方式”

响应处理参数用于配置服务端响应数据的接收与解析规则，包括responseType、success、fail等，决定请求结果的处理逻辑。

1. responseType：响应数据类型（可选，默认text）

指定响应数据的原始类型，与dataType的区别在于：responseType控制“原始数据格式”，dataType控制“解析后的格式”。

核心取值与场景

- `text`（默认）：响应数据为字符串，后续可通过dataType解析为JSON或直接使用。
- `arraybuffer`：响应数据为二进制数组（ArrayBuffer），适用于文件下载、图片获取等场景。
- `blob`（仅H5端支持）：响应数据为Blob对象，适用于H5端文件下载。

实战示例：下载图片并转为本地路径

```

1  // 1. 以arraybuffer接收图片二进制数据
2  uni.request({
3    url: 'https://api.example.com/image/123',
4    responseType: 'arraybuffer',
5    success: (res) => {
6      // 将ArrayBuffer转为Base64
7      const base64 = uni.arrayBufferToBase64(res.data)
8      // 转为本地图片路径 (可用于image标签展示)
9      const imageUrl = `data:image/png;base64,${base64}`
10     console.log('图片路径', imageUrl)
11   }
12 })

```

2. 回调函数参数：success/fail/complete

用于处理请求结果，是请求生命周期的核心回调，三类回调的定位与执行逻辑如下：

- **success**：请求成功（HTTP状态码200~299）时执行，参数res包含响应头、响应数据等。
- **fail**：请求失败（网络错误、HTTP状态码≥400）时执行，参数err包含错误码、错误信息。
- **complete**：无论成功失败都执行，用于关闭加载动画、释放资源等通用逻辑。

实战示例：完整回调流程

Code block

```

1  uni.request({
2    url: 'https://api.example.com/goods/list',
3    data: { categoryId: 1 },
4    success: (res) => {
5      // 先判断HTTP状态码，再判断业务状态码
6      if (res.statusCode === 200 && res.data.code === 0) {
7        console.log('商品列表', res.data.data)
8      } else {
9        uni.showToast({ title: res.data.msg || '数据获取失败', icon: 'error' })
10     }
11   },
12   fail: (err) => {
13     // 根据错误码给出具体提示
14     let errMsg = '请求失败，请重试'
15     if (err.errCode === 1001) errMsg = '无网络，请检查网络设置'
16     if (err.statusCode === 404) errMsg = '接口不存在'
17     uni.showToast({ title: errMsg, icon: 'error' })
18   },
19   complete: () => {
20     // 关闭加载动画 (无论成败都执行)
21     uni.hideLoading()

```

```
22     }  
23   })
```

五、安全与超时参数：保障请求的“稳定性与安全性”

安全与超时参数用于控制请求的生命周期和安全策略，包括timeout、sslVerify等，是提升请求稳定性的关键。

1. timeout：请求超时时间（可选，默认60000ms）

设置请求的超时时间（单位：毫秒），超过该时间未收到响应则触发fail回调，避免请求长期阻塞。

实战注意事项

- 默认超时时间为60秒，可根据接口响应速度调整（如高频接口设为10秒，文件上传接口设为30秒）。
- 超时后fail回调的errCode为1004，可据此判断并提示“请求超时”。
- 小程序端部分平台（如微信）对超时时间有上限限制（如300秒），超过上限会被强制截断。

实战示例

Code block

```
1  // 设置10秒超时的请求  
2  uni.request({  
3    url: 'https://api.example.com/goods/list',  
4    timeout: 10000, // 10秒超时  
5    fail: (err) => {  
6      if (err.errCode === 1004) {  
7        uni.showToast({ title: '请求超时，请检查网络或重试', icon: 'error' })  
8      }  
9    }  
10  })
```

2. sslVerify：SSL证书验证（可选，默认true）

控制是否验证服务端SSL证书，仅在App端和微信小程序（基础库2.22.0+）支持，用于适配自签名证书的服务端。

核心场景与风险

- `true`（默认）：验证SSL证书，保障请求安全，避免中间人攻击。

- `false`：不验证SSL证书，适用于开发环境（如自签名证书的测试服），生产环境禁用，存在安全风险。

实战示例：开发环境适配自签名证书

Code block

```
1 // 仅在开发环境关闭SSL验证（生产环境需改为true）
2 const isDev = process.env.NODE_ENV === 'development'
3 uni.request({
4   url: 'https://dev-api.example.com/user/info',
5   sslVerify: !isDev, // 开发环境false，生产环境true
6   success: (res) => {
7     console.log('用户信息', res.data)
8   }
9 })
```

3. withCredentials：跨域请求携带Cookie（可选，默认false）

控制跨域请求时是否携带Cookie，仅H5端支持，用于基于Cookie的身份验证场景。

实战注意事项

- 设置为true时，服务端需配置 `Access-Control-Allow-Credentials: true`，否则浏览器会拦截响应。
- 小程序端和App端跨域请求会自动携带Cookie，无需配置该参数。

实战示例：H5端跨域携带Cookie

Code block

```
1 // H5端跨域请求携带Cookie
2 uni.request({
3   url: 'https://cross-domain-api.example.com/user/info',
4   withCredentials: true,
5   success: (res) => {
6     console.log('用户信息', res.data)
7   }
8 })
```

六、多端适配与避坑要点

1. 各平台参数差异

平台	参数差异点	适配方案
微信小程序	1. 不支持withCredentials； 2. sslVerify需基础库2.22.0+；3. 请求域名需在小程序后台配置白名单	1. 基于Cookie的验证改用token；2. 配置request合法域名；3. 低版本基础库避免使用sslVerify
App端	1. 支持sslVerify和自定义证书；2. 超时时间可设置更长（如300秒）；3. 支持本地代理配置	1. 生产环境开启sslVerify；2. 大文件上传设置较长超时时间；3. 通过 plus.net 配置代理
H5端	1. 支持withCredentials和blob响应类型；2. 跨域需服务端配置CORS；3. 部分浏览器限制自定义请求头	1. 跨域携带Cookie时配合服务端CORS配置；2. 避免使用浏览器禁用的请求头字段

2. 常见参数配置错误与解决方案

2.1 问题1：POST请求数据服务端无法解析？

原因：Content-Type与data格式不匹配（如用application/json格式传递表单数据）。

解决方案：根据服务端要求配置Content-Type，如服务端接收表单数据则设置：

Code block

```
1  header: {  
2    'Content-Type': 'application/x-www-form-urlencoded'  
3  }
```

2.2 问题2：跨域请求时token无法传递？

原因：H5端跨域请求时，服务端未配置Access-Control-Allow-Headers允许Authorization字段。

解决方案：服务端响应头添加：

Code block

```
1  Access-Control-Allow-Headers: Authorization, Content-Type
```

2.3 问题3：响应数据解析为[object Object]但无法获取字段？

原因：服务端返回的是JSON字符串，但dataType未设为json，导致解析失败。

解决方案：显式设置dataType为json，或手动解析：

```
1  success: (res) => {  
2    const data = JSON.parse(res.data) // 手动解析JSON字符串  
3    console.log(data.name)  
4  }
```

七、总结

uni.request的参数配置需遵循“场景匹配、格式对齐、多端适配”三大原则，核心要点如下：

1. **基础配置精准化**：url使用绝对路径保障兼容性，method根据业务场景选择，避免滥用POST。
2. **请求内容规范化**：data格式与Content-Type严格匹配，header携带必要的验证信息与元数据。
3. **响应处理全面化**：通过success/fail/complete覆盖请求全流程，结合dataType和responseType适配不同响应格式。
4. **安全超时合理化**：生产环境开启sslVerify，根据接口类型设置合理超时时间，H5端跨域注意Cookie携带规则。

实际开发中，建议封装统一的请求工具类，预设通用参数（如header、timeout），简化重复配置，同时针对不同平台做差异化处理，提升开发效率与请求稳定性。

（注：文档部分内容可能由 AI 生成）